BOOTCAMP III | DESAFIO FINAL

Relatorio Tecnico de Governanca e Etica

TaskAnalyzer - DevOps, testes automatizados, containerizacao e governanca de IA

Aluno	Caio de Almeida Perez
Repositorio	https://github.com/Caio-Perez33/sdd-task-analyzer
Finalidade	Registrar arquitetura, validacao tecnica, riscos de uso de IA e controles de homologacao humana da Etapa Final.

1. Resumo executivo

O TaskAnalyzer transforma uma lista de tarefas em indicadores de produtividade. A Etapa Final organiza a validacao em um pipeline CI/CD, padroniza a execucao dos testes em container e registra como o contrato SDD e as regras de contexto limitam o uso de IA. A solucao procura ser simples, auditavel e reproduzivel, sem persistir dados e sem dependencias externas de runtime.

Estado de validacao deste pacote

Verificacao	Estado observado	Limite da verificacao
Testes pytest	8 testes aprovados neste ambiente de preparacao.	Executar novamente no repositorio local antes do commit.
GitHub Actions	Workflow configurado no pacote.	O status remoto so pode ser confirmado apos push/PR no GitHub.
Docker / Compose	Dockerfile e docker-compose.yml preparados.	O motor Docker nao esta disponivel neste ambiente de preparacao; executar no Docker Desktop local.
Pitch em video	Roteiro pronto em docs/.	Gravacao, publicacao e URL real dependem do aluno.

2. Arquitetura e controles de entrega

A arquitetura possui cinco partes: modulo funcional em src/task_analyzer.py; Test Harness em tests/test_harness.py; contrato e regras de contexto em specs/ e CONTEXT_RULES.md; workflow de CI em .github/workflows/ci.yml; e ambiente de teste isolado em Dockerfile e docker-compose.yml.

- CI: testa no Python 3.11 em push e pull request. A construcao da imagem depende do sucesso dos testes.
- Container: usa imagem Python slim, usuario nao-root e sistema de arquivos somente leitura na execucao do servico Compose.
- Dependencias: requirements.txt declara pytest apenas para teste; requirements.yaml registra que o runtime nao exige dependencias externas.
- Reprodutibilidade: o comando Docker Compose executa a mesma suite de testes em um ambiente isolado.

3. Analise de alucinacoes e aderencia ao contrato

As regras de contexto funcionam como limites verificaveis para a geracao por IA. O contrato em specs/task_analyzer_spec.md define campos, valores enumerados, validacao temporal, metricas e unidades. Os testes convertem casos importantes em verificacoes executaveis.

Risco analisado	Controle do contrato	Como verificar
A IA sugere pandas ou outra dependencia nao autorizada.	Runtime sem bibliotecas externas; requirements.yaml documenta o limite.	Revisar requirements.txt e o diff do PR.
A IA cria banco de dados ou grava arquivos.	CONTEXT_RULES proibe persistencia. O modulo calcula somente sobre a entrada.	Inspecionar imports, chamadas de arquivo e alteracoes no diff.
A formula de atraso usa todas as tarefas como denominador.	Contrato determina usar apenas concluidas como base da taxa.	Testar tarefa atrasada e tarefa nao concluida no cenario de sucesso.
A media divide por zero sem tarefas concluidas.	Media e taxa de atraso retornam 0.0 quando nao ha concluidas.	Testes de lista vazia e tarefas apenas pendentes.
A IA aceita cronologia inconsistente ou muda silenciosamente a regra.	TaskValidationError, testes de data e decisao documentada de refinamento do contrato.	Executar os testes de datas e comparar o comportamento com a especificacao.

Os itens acima sao riscos avaliados a partir do contrato e do codigo; nao sao apresentados como incidentes historicos de IA que necessariamente tenham ocorrido.

4. Homologacao humana (Critical-in-the-Loop)

A revisao humana e a ultima barreira antes do merge. A pessoa responsavel deve confirmar que a implementacao respeita as regras SDD, que os testes cobrem os comportamentos e que nenhuma alteracao inesperada entrou no pull request. O fato de um trecho ter sido produzido por IA nao substitui essa verificacao.

- Conferir a assinatura publica analyze_tasks(tasks) e os campos de entrada/saida.
- Conferir validacao de IDs, prioridades, status, timezone e ordem cronologica.
- Conferir unidades e arredondamento: minutos, horas e percentual de atraso.
- Revisar os arquivos de dependencias, o workflow e o Dockerfile.
- Executar pytest, Docker Compose e revisar o resultado real em GitHub Actions.
- Revisar o diff antes de aprovar o PR; integrar somente se os controles passarem.

5. Sustentabilidade, privacidade e etica

- Minimizacao de dependencias reduz complexidade de instalacao e manutencao.
- O processamento e em memoria e nao grava dados da tarefa em arquivo ou banco.
- A imagem slim evita componentes que nao sao necessarios para a suite de testes.
- A execucao como usuario nao-root e o filesystem somente leitura reduzem privilegios do processo de teste.
- O relatorio distingue resultados verificados de etapas que dependem do ambiente GitHub/Docker do aluno.

6. Resultado dos testes automatizados

A suite inclui oito casos: sucesso com metricas gerais e por prioridade; conclusao anterior a criacao; entrada vazia; apenas tarefas pendentes; status invalido; ID duplicado; tarefa concluida sem datas de execucao; e conclusao anterior ao inicio. O pacote foi executado neste ambiente de preparacao, com oito testes aprovados. A execucao deve ser repetida no repositorio local e em Docker antes de considerar a entrega totalmente homologada.

Comandos de verificacao

```
python -m pip install -r requirements.txt
python -m pytest -v
docker compose run --build --rm testes
```

7. Conclusao e checklist de submissao

A confiabilidade do TaskAnalyzer resulta da combinacao entre contrato SDD, testes, automacao de CI, ambiente Docker padronizado e revisao humana. A solucao esta estruturada para permitir que as mesmas verificacoes sejam repetidas localmente e no GitHub, com resultados observaveis.

Antes de enviar para a plataforma AVA, confirme todos os itens abaixo:

- [ ] O repositorio publico contem os arquivos desta etapa e o PR foi revisado.
- [ ] O workflow CI TaskAnalyzer foi executado no GitHub e terminou em verde.
- [ ] `docker compose run --build --rm testes` terminou com oito testes aprovados.
- [ ] O video foi gravado na horizontal, com audio e camera visiveis, e tem aproximadamente cinco minutos.
- [ ] O video foi publicado como Publico ou Nao listado e a URL real foi copiada para a entrega.
- [ ] O relatorio foi relido depois das verificacoes locais para garantir que nao afirma uma aprovacao ainda pendente.

Registro de honestidade da verificacao

Este PDF nao declara que o GitHub Actions foi executado remotamente nem que a imagem Docker foi executada neste ambiente. Essas duas evidencias precisam ser confirmadas pelo responsavel depois de enviar a branch ao GitHub e executar o Docker Desktop. O roteiro de video acompanha o pacote em docs/Roteiro_Pitch_Etapa_Final.md.

Documento de apoio para a entrega final do Bootcamp III. Atualize-o caso os resultados reais de CI, Docker ou a configuracao da entrega mudem.